NKT
Tag 5: Fragen und Wünsche

Tobias Wrede, Alexa Katharina-Ronja Brockmöller

13. März 2026

Gliederung

1. Classpath und Package-Pfad
2. Higher Order Functions und Streams
3. Heap-Aufgabe

Gliederung

1. Classpath und Package-Pfad
2. Higher Order Functions und Streams
3. Heap-Aufgabe

Classpath und Package-Pfad

Heinrich Heine Universität Düsseldorf

Wir haben die java-Datei

`/home/marlin/progra/de/hhu/progra/Student.java`

mit folgendem Anfang:

Student.java

```java
package de.hhu.progra;

public class Student {
```

Welcher Teil vom Dateipfad ist der Classpath, welcher der Package-Pfad?

Classpath und Package-Pfad

Heinrich Heine Universität Düsseldorf

Wir haben die java-Datei

`/home/marlin/progra` / `de/hhu/progra` `/Student.java`

mit folgendem Anfang:

Student.java

```java
package de.hhu.progra;

public class Student {
```

Welcher Teil vom Dateipfad ist der Classpath, welcher der Package-Pfad?

Classpath und Package-Pfad

Wir haben die java-Datei

`/home/marlin/progra` / `de/hhu/progra` `/Student.java`

mit folgendem Anfang:

Student.java

```java
package de.hhu.progra;

public class Student {
```

Welcher Teil vom Dateipfad ist der Classpath, welcher der Package-Pfad?

Was muss ich beachten, wenn ich in der Klasse `de.hhu.Verwaltung` die Klasse `de.hhu.progra.Student` verwenden möchte?

Gliederung

1 Classpath und Package-Pfad

2 Higher Order Functions und Streams

3 Heap-Aufgabe

Higher Order Functions

Heinrich Heine Universität Düsseldorf

Higher Order Function (HOF):

- Eine Funktion, die andere Funktionen als Parameter entgegennehmen oder Funktionen zurückgeben
- Beispiele: map,forEach auf Collections
- Datentyp von Funktionen sind funktionale Interfaces
- Funktionale Interfaces: Interfaces mit genau einer abstrakten Methode
- Für Funktionen mit einem Parameter und einem Rückgabewert gibt es das funktionale Interface *Function<E,A>*:

```
interface Function<E,A> {
A run(E input);
}
```

Lambda-Ausdrücke

- Kurzschreibweise einer Funktion, um diese leicht als Parameter zu übergeben
- Syntax: (Parameter) -> {Anweisungen}

```
Function<Integer, String> f1 = (Integer n) -> {
return "#" + n;
};
```

Streams

- Datenstrom
- Lässt sich bei Collections durch Aufruf von *stream()* erzeugen
- verschiedene Operationen auf Streams:
  - *map*: Wendet Funktion auf jedes Element an
  - *forEach*: Beendet Stream
  - *filter*: Filtert Stream nach einer Bedingung
  - *toList*: Erzeugt Liste
- sind idiomatisch außer:
  - Collection wird verändert
  - zu große Labda-Ausdrücke: stattdessen Methode erstellen
  - Seiteneffekte außerhalb von forEach

Gliederung

1. Classpath und Package-Pfad
2. Higher Order Functions und Streams
3. Heap-Aufgabe

Heap-Aufgabe

Heinrich Heine Universität Düsseldorf

```
Aufgabe 6                                                ____ / 8 Punkte

Der Konstruktor der Klasse Finder speichert Kopien der übergebenen Elemente
in einer neuen ArrayList (Shallow Copy) und sortiert sie. Die Instanzmethode
exists kann benutzt werden, um mithilfe von binärer Suche Elemente in der
gespeicherten Liste zu suchen.

Finder.java
---------------------------------------
import java.util.ArrayList;
import java.util.Collections;

class Finder<T extends Comparable<T>> {
    private ArrayList<T> elements;

    Finder(ArrayList<T> elements) {
        this.elements = new ArrayList<T>(elements);
        Collections.sort(this.elements);
    }

    boolean exists(T needle) {
        return Collections.binarySearch(this.elements, needle) >= 0;
    }
}
```

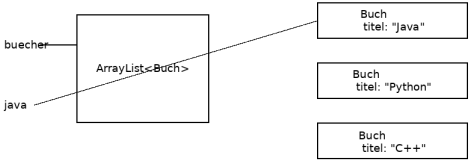
Heap-Aufgabe

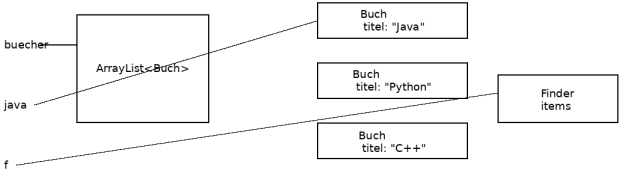
Heinrich Heine Universität Düsseldorf

```
Demo.java
--------------------------------------
import java.util.ArrayList;

static void main(String[] args) {
    ArrayList<Buch> buecher = new ArrayList<>();
    Buch java = new Buch("Java");
    buecher.add(java);
    buecher.add(new Buch("Python"));
    buecher.add(new Buch("C++"));

    Finder<Buch> f = new Finder<>(buecher);
    System.out.println(f.exists(new Buch("Python"))); // true
    System.out.println(f.exists(new Buch("C++")));    // true
    System.out.println(f.exists(new Buch("Java")));   // true
    System.out.println(f.exists(java));               // true
    System.out.println(f.exists(new Buch("Rust")));   // Aufgabenteil a)

    buecher.set(0, new Buch("A"));
    System.out.println(f.exists(new Buch("A")));      // Aufgabenteil c)
}

Buch.java
--------------------------------------
class Buch implements Comparable<Buch> {
    private String titel;

    Buch(String titel) {
        this.titel = titel;
    }
```

Heap-Aufgabe

Heap nach Ausführen von Zeile 8 (Vorlage)



Heap-Aufgabe

Heap nach Ausführen von Zeile 10 (Vorlage)